The National Higher School of Artificial Intelligence

Introduction to AI Course Project

Spring 2026

Algiers in 24 Hours:

A Time-Constrained Orienteering Approach to Tourist Route Optimisation

Full Name Group

Abstract

This report studies algorithmic approaches for the Orienteering Problem with Time Windows (OPTW) applied to single-day touristic route planning in Algiers. Eight solution methods are implemented and compared: a Greedy construction heuristic with two priority criteria, Simulated Annealing with Boltzmann and Cauchy acceptance criteria and two cooling schedules, Tabu Search augmented with strategic oscillation, GRASP with a duration-aware scoring function, two Genetic Algorithm variants including a feasibility-preserving tailored GA based on the MaxShift operator, and an exact Mixed-Integer Linear Program solved by CPLEX. Experiments are conducted on a curated Algiers landmark dataset of approximately 59 sites and on standard Solomon TOPTW benchmark instances of 50 and 100 nodes, spanning clustered, random, and mixed instance classes. Methods are evaluated with respect to wall-clock time, collected interest score, optimality gap, and optimal-finding rate.

The comparative study characterises the quality–time tradeoffs of these methods across instance classes and problem scales, and identifies solver behaviours that are sensitive to cooling schedule design, crossover and repair strategies, and neighbourhood filtering mechanisms. A central finding is that feasibility-aware operators are critical for population-based search under tight temporal constraints, and that advanced local-search metaheuristics and tailored genetic operators consistently outperform naïve greedy constructions and unconstrained GA designs within realistic time budgets. Exact methods remain valuable as calibration and upper-bound tools but scale poorly beyond small instances. A lightweight web demonstrator exposes the implemented models and solvers, allowing users to specify their journey parameters and visualise the resulting optimised itinerary on a map of Algiers.

1. Introduction

The Traveling Salesman Problem (TSP) is one of the most extensively studied problems in combinatorial optimization. Given a set of cities and pairwise travel costs, the objective is to determine a minimum-cost Hamiltonian cycle visiting each city exactly once before returning to the starting point. The TSP is known to be NP-hard, as it can be related through polynomial reductions to the Hamiltonian Cycle problem.

Beyond its theoretical importance, the TSP constitutes the foundation of numerous routing and scheduling problems in operations research. Consequently, many variants have emerged in the literature to model additional real-world constraints such as capacities, profits, precedence relations, and time windows.

The Orienteering Problem (OP) is a variant of the TSP in which each vertex is associated with a profit or score. Unlike the classical TSP, the OP seeks to determine a subset of vertices maximizing the collected score while satisfying a constraint on the maximum allowable tour length. The problem was first introduced by Tsiligirides, where it was referred to as the Generalized Traveling Salesman Problem (GTSP).

In this project, the Orienteering Problem with Time Windows (OPTW) is considered, in which a time interval $[O_i, C_i]$ is associated with each vertex. The objective is identical to the OP, with the additional constraint that each vertex must either be visited within its time window or skipped entirely.

This project applies the OPTW to the practical problem of touristic landmark visit planning in the city of Algiers. Given the density and diversity of Algiers' cultural and historical sites — many of which have restricted visiting hours — the OPTW provides a natural and expressive model for planning a day's itinerary that maximises the interest of visited landmarks while respecting opening times and the tourist's available time budget. The system takes as input a set of landmarks with associated interest scores, opening hours, and visit durations, and produces an ordered daily route starting and ending at the tourist's hotel. A demonstration interface was developed to allow users to specify their starting time, available duration, and travel day, and to visualise the resulting optimised route on a map of Algiers. The practical benefit to the end user is a ready-to-follow itinerary that avoids wasted travel and closed sites, making the most of a limited visit to the city.

2. State of the Art

Exact Methods

One of the earliest exact approaches proposed for solving the OP is the Branch-and-Cut algorithm introduced by Fischetti et al.[3], which combines linear programming relaxation, branching, and cutting planes to reduce the search space. Although effective on small and medium-sized instances, its computational cost increases rapidly with the number of vertices. This motivated the development of heuristic approaches, notably those of Tsiligiridis[20] and Golden and Levy[5], to obtain near-optimal solutions more efficiently.

For the OPTW, Kantor and Rosenwein[7] proposed a heuristic tree-search algorithm that incrementally constructs routes by adding temporally feasible vertices while continuously verifying time-window constraints. Although effective only on small instances, the method highlighted the strong dependency between vertices induced by temporal constraints. Righini and Salani later[14] proposed an exact approach reformulating the OPTW as a Resource Constrained Elementary Shortest Path Problem (RCESPP), solved via dynamic programming with Decremental State Space Relaxation (DSSR). Despite significantly reducing the number of explored states through dominance and relaxation strategies, the method still suffers from state-space explosion on large instances and remains sensitive to the structure of the time windows.

Metaheuristic Methods

Classical local search methods remained limited by their tendency to become trapped in local optima, motivating the development of more advanced metaheuristics capable of balancing intensification and diversification. Among the most influential approaches for the OPTW is the Variable Neighborhood Search (VNS) introduced by Labadie et al.[10], which alternates between several neighborhood structures while perturbing and reconstructing the current solution to escape local minima. Although the method demonstrated strong performance and hybridization potential, it remains sensitive to perturbation strength and parameter tuning.

Tabu Search, proposed by Archetti et al.[1], incorporates adaptive memory structures to prevent revisiting previously explored solutions and reduce cycling behavior. The method achieves effective intensification and broad exploration, though its performance strongly depends on the configuration of tabu tenure and diversification strategies. Simulated Annealing (SA) has also been widely applied to the OPTW; inspired by thermodynamic processes, it probabilistically accepts deteriorating solutions to escape local optima, gradually shifting from exploration to exploitation as the temperature decreases. Its effectiveness, however, remains highly dependent on the cooling schedule and parameter configuration.

The Greedy Randomised Adaptive Search Procedure (GRASP), introduced by Feo and Resende[2], combines a randomised greedy construction phase with an iterative local search phase. At each iteration, a candidate solution is built by selecting elements from a Restricted Candidate List (RCL) that balances quality and randomness, and is then improved by local search; the best solution across all iterations is returned. GRASP has been applied to several routing problems due to its simplicity and its ability to explore diverse regions of the solution space without maintaining a population, making it a lightweight alternative to population-based metaheuristics.

Population-Based and Learning-Based Methods

Karbowska-Chilińska and Zabielski[8] proposed a Genetic Algorithm (GA) that evolves a population of candidate routes through crossover and mutation operators. Although GAs provide strong exploration capabilities, they often struggle with feasibility preservation under time-window constraints, sensitivity

to parameter tuning, and premature convergence. Ant Colony Optimization (ACO), proposed by Montemanni and Gambardella[12], incrementally constructs routes using pheromone-guided probabilistic decisions, demonstrating strong constructive and cooperative search capabilities while remaining sensitive to parameterization and premature convergence toward suboptimal solutions.

More recent approaches have explored Reinforcement Learning (RL) techniques, learning routing policies directly from interactions with the optimization environment. Methods such as Pointer Networks and attention-based models trained with policy gradient algorithms have been applied to variants of the orienteering problem, enabling end-to-end learning of construction heuristics without explicit problem-specific engineering. Although promising, such methods generally require substantial computational resources, large training datasets, and still lack the robustness and constraint-handling precision of established hybrid metaheuristics on tightly constrained instances such as the OPTW.

This Work

In this report, we present a comparative study of several algorithms for solving the OPTW in the context of touristic landmark visit planning in Algiers. The study focuses on implementing heuristic, metaheuristic, and exact approaches while introducing enhancements intended to overcome some of their known limitations. Specifically, we implement a Greedy algorithm extended with a profit-to-time-commitment ratio criterion that accounts for both travel time and visit duration, avoiding the over-selection of long-duration landmarks that standard greedy heuristics exhibit. We implement a Simulated Annealing algorithm comparing two cooling schedules (exponential and linear) alongside two acceptance criteria (Boltzmann and Cauchy), with a reheating mechanism to escape temperature collapse. We implement a Genetic Algorithm based on Order Crossover (OX) and introduce a tailored feasibility-preserving genetic operator based on the MaxShift quantity, which enables crossover and insertion moves to be validated without full route re-simulation. We also implement the CPLEX exact MILP solver with an MTZ-based time-propagation formulation and multi-slot time-window constraints, providing optimality certificates as an upper-bound reference. The Tabu Search is augmented with a successive filtering mechanism based on weakness and fitness scores to keep iterations tractable, and with strategic oscillation to systematically escape plateaus by traversing infeasible corridors between feasible solution clusters. Finally, the GRASP metaheuristic is implemented with a scoring function that penalises visit duration in addition to travel time, and a local search phase using prioritised Replace, Insert, and Swap operators with first-improvement Swap to avoid worst-case blow-up. The proposed methods were evaluated using both locally generated datasets and the benchmark dataset of Montemanni and Gambardella commonly used in the OPTW literature.

3. Dataset Building

3.1 Data Collection

We created a dataset of 58 tourist landmarks in Algiers, classified into five landmark categories: historical, attractions, traditional and artistic, shopping, and religious sites.

Each entry in the dataset has GPS coordinates, the time the site is open to visitors, an estimate for how long it takes to visit, and a user rating from either Google Maps or TripAdvisor. Public sources were used to check the hours of operation, although much of the data was obtained from Google Maps directly. Table 1 presents the attributes, attribute types, and methods used to obtain the data.

Table 1: Dataset attribute descriptions.

Attribute	Type	Source	Description
Name	Text	Manually	Full official name of the landmark
Category	Text	Manually	Thematic group assigned to one of five classes
Latitude	Float	Google Maps	WGS84 decimal degrees, manually verified
Longitude	Float	Google Maps	WGS84 decimal degrees, manually verified
Opening hours	Text	Official listing	Structured time-window per day (e.g. 09:00–19:00)
Visit duration	Integer	Estimation	Expected time on-site in minutes
Google rating	Float	Google Maps	Aggregated user score on a 1–5 scale
TripAdvisor rating	Float	TripAdvisor	Aggregated user score on a 1–5 scale (missing for 18 sites)
Interest Score	Float	Computed	Google + TripAdvisor, or Google $\times$ 2 when TripAdvisor is missing, mapped to 1–10

3.2 Preprocessing

- **Coordinate verification:** All GPS coordinates were manually cross-checked using Google Maps satellite view to avoid positioning errors.
- **Interest score calculation:** An overall Interest Score was calculated on a 1–10 scale. When both ratings are available, the score is defined as Google + TripAdvisor; when only a Google rating exists (18 landmarks, 31%), the score is set to Google $\times$ 2, maintaining scale consistency.
- **Category assignment:** Each landmark was classified into one of five categories: historical, attraction, tradition & art, religious, and shopping. This enables interest-based filtering in the Day Planner application and improves the tourist experience.
- **Opening-hours standardisation:** Time windows used by the optimiser were converted into structured formats (e.g. 09:00–17:00). For venues with split opening hours (for example, 11:00–12:30 and 15:00–17:30), multiple time windows were defined for each day.

3.3 Descriptive Statistics and Characteristics of the Dataset

The dataset contains 58 unique landmarks divided into five categories. Numerical descriptive statistics are shown in Table 2, while the descriptive visualisations are shown in Figure 1 and Figure 2. Because two landmarks were marked as continuously closed, the visualisations are based only on the remaining 56 landmarks that are available to the planner.

The numerical summary is based on variables that directly influence recommendation and route optimisation: the calculated Interest Score, the two external rating sources, and the estimated visit duration. These statistics are used to measure the quality of the selected landmarks and the practicality of including them in daily itineraries.

Table 2: Descriptive statistics for numerical attributes.

Metric	Interest score	Google	TripAdvisor	Duration (h)
n	58	58	40	57
Mean	8.376	4.266	4.123	1.544
Median	8.600	4.400	4.200	1.500
Std dev	0.853	0.412	0.488	0.836
Min	5.600	3.000	2.000	0.500
Max	10.000	5.000	4.800	3.000

The Interest Scores range from 5.6 to 10.0 with an average of 8.38. This means that the data set is generally of high quality with good reviews. The Google and TripAdvisor ratings are highly correlated (averages of 4.27 and 4.12 respectively) which validates the summation-based scoring formula. The average visit duration of 1.54 h per landmark is directly fed to the optimiser when constructing feasible itineraries within a fixed time budget.

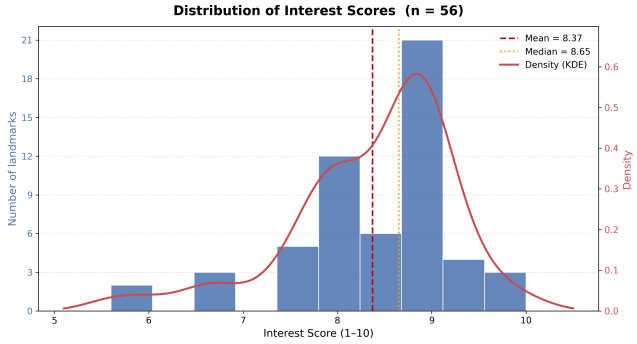

Figure 1: Distribution of Interest Scores with KDE overlay. Dashed = mean; dotted = median.

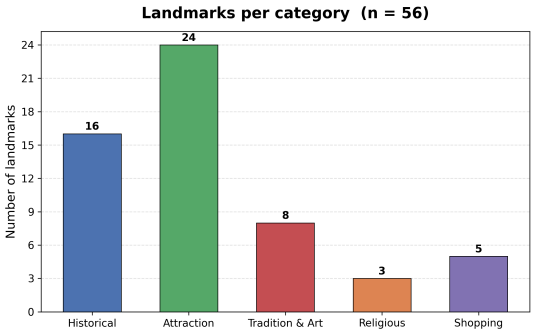

Figure 2: Landmark count per category for the 56 available landmarks.

4. Problem Solving Techniques

Problem Formulation

This project addresses the Orienteering Problem with Time Windows (OPTW) in the context of touristic landmark visit planning. The problem is defined on a complete undirected graph $G = (L, E)$, where each vertex $l_i \in L$ represents a landmark and each edge $(l_i, l_j) \in E$ is associated with a non-negative travel time t_{ij}. Each landmark $i \in L$ is characterised by a profit p_i, a service duration v_i, and a time window $[O_i, C_i]$ defining the feasible arrival interval. A special vertex 1 represents the hotel and serves as both the starting and ending point of the route, with $p_1 = 0$, $v_1 = 0$, and $[O_1, C_1] = [0, T]$, where T denotes the maximum allowed route duration.

The objective is to determine a feasible path P — an ordered sequence of vertices starting and ending at the hotel — maximising the total collected profit $\max \sum_{i \in P} p_i$, subject to the temporal feasibility constraints $O_i \leq arr_i \leq C_i$, where arr_i denotes the arrival time at vertex i. After arriving, the service duration v_i must also complete within the associated time window. A solution is valid if all visited landmarks satisfy their time-window constraints while respecting the maximum route duration T. Since landmark availability may vary by day, the formulation incorporates day-dependent time windows. The user may also specify a custom journey start time. Travel times between landmarks are computed from geographical coordinates (latitude and longitude) assuming a constant travel speed of 25 km/h.

Greedy Algorithm

The Greedy solver constructs a feasible tour incrementally starting from the hotel vertex l_1. At each step it evaluates the feasible candidate set $F = \{l_i \in L \mid \text{route feasibility is preserved}\}$ and selects the next landmark according to one of two priority criteria: $\max(p_i)$, which prioritises landmarks with the highest profit, or $\max(p_i/t_{ci})$, where t_{ci} denotes travel time from the current vertex c to landmark i, introduced to balance landmark attractiveness against travel cost. After every insertion, arrival times and the remaining time budget are updated to verify both the time-window constraint $O_i \leq arr_i \leq C_i$ and the return-feasibility condition $arr_i + v_i + t_{i1} \leq T$, ensuring the route remains temporally feasible and can still return to the hotel within the maximum duration.

Simulated Annealing

SA is initialised from the greedy solution rather than a random tour. Since SA's exploration capability is highest at the beginning when temperature is high, starting from a reasonable solution allows the algorithm to spend that exploration budget in a productive region of the search space rather than recovering from a poor initialisation.

Perturbation operators

At each iteration one of four operators is selected uniformly at random and applied to a copy of the current tour: **Swap** exchanges the positions of two randomly chosen landmarks; **Insert** adds a randomly selected feasible unvisited landmark at a random position; **Remove** deletes a randomly chosen landmark; **Invert** reverses the subsequence between two randomly chosen indices. If the resulting neighbour is infeasible (violates time windows or the budget), it is rejected outright and the iteration continues without a temperature update. This strict feasibility filter means SA only ever moves between valid tours, at the cost of occasionally wasting an iteration. The Invert operator was included because reordering a subsequence can resolve time-window conflicts without changing which landmarks are visited, a move that neither Swap nor Remove can achieve alone.

Acceptance criteria

If the neighbour improves the objective ($\Delta = f(P') - f(P) > 0$) it is accepted directly. Otherwise it is accepted with probability $e^{\Delta/T}$ under the Boltzmann criterion, or $1/(1+\Delta^2/T)$ under the Cauchy criterion (note $\Delta < 0$ in both cases). Boltzmann decays exponentially with the magnitude of deterioration; Cauchy uses a squared penalty in the denominator, making it more tolerant of small deteriorations but much stricter about large ones. The best solution seen across all accepted states is tracked separately and returned at the end, so a bad acceptance move cannot overwrite the global best.

Cooling schedules

Two strategies are implemented: *exponential* $T_{k+1} = \alpha \cdot T_k$ with $\alpha \in (0,1)$, and *linear* $T_{k+1} = \max(T_k - \alpha, 0)$. Exponential cooling reduces temperature multiplicatively, spending more iterations at high temperatures and converging slowly at the end — appropriate when broad exploration is more valuable than fine-grained local refinement. Linear cooling reduces temperature at a constant rate, allocating the iteration budget more uniformly across temperature levels.

Reheating

When $T < 10^{-8}$ the search has effectively become a pure greedy descent with no acceptance of worse solutions. Rather than terminating, a reheating mechanism restores the temperature to $r_{\text{reheat}} \times T_0$. Setting $r_{\text{reheat}} = 0$ disables reheating entirely (single-run annealing); setting it to 1 fully resets the temperature, equivalent to a multi-start strategy where each cooling cycle is a fresh annealing run from the current state. Intermediate values implement partial reheating, restoring some exploration capability without resetting the search to its initial intensity.

Genetic Algorithm

Two variants are implemented: GENETICSOLVER, which repairs infeasible individuals post-hoc, and TAILOREDGENETICSOLVER, which maintains feasibility by construction through an augmented route representation.

Fitness functions

GENETICSOLVER uses total route profit $\sum_{i \in P} p_i$ as fitness, in combination with `_resolve_tour` which corrects infeasibility structurally after each mutation so the fitness only needs to rank feasible routes by collected profit. TAILOREDGENETICSOLVER uses $F(P) = \sum_{i \in P} p_i + (T - D(P))/T$, where the time-efficiency bonus $(T - D(P))/T$ rewards routes that finish with slack. Without it, two routes with identical visited sets but different orderings score identically, removing all pressure toward compact routes; in practice omitting it caused the tailored solver to produce zigzag routes that consumed the budget so tightly that mutation could never insert new landmarks in later generations.

Augmented representation and MaxShift

The tailored solver annotates each position i in the route with $(arr_i, wait_i, start_i, end_i, MaxShift_i)$. $MaxShift_i$ is computed in a backward pass and represents the maximum delay that can be applied to $start_i$ without violating any downstream time window or the budget: $MaxShift_i = \min(C_i - start_i - v_i,\ wait_{i+1} + MaxShift_{i+1})$. For the last landmark the downstream term is $T - D(P)$. For landmarks with multiple time slots $[O_k, C_k]$, the closing slack of each slot containing the current visit is computed and the largest valid

one is kept, so the tightest constraint between the landmark's own closing time and what downstream can absorb is always used.

Selection

Tournament selection draws $k = \min(3, |P_t|)$ individuals at random and returns the two best. Three alternatives were implemented and tested — random, probability-biased, and fitness-proportionate (roulette wheel) — but tournament selection produced consistently better results and is used as the default in both solvers.

Crossover

GENETICSOLVER uses order crossover: a random segment $[a, b]$ is copied from the shorter parent; remaining positions are filled from the longer parent in order, skipping duplicates; two children are produced symmetrically. TAILOREDGENETICSOLVER validates each candidate splice: a cut at position i in $P^{(1)}$ into position j in $P^{(2)}$ is valid if $end_i^{(1)} + t_{l_i^{(1)}, l_j^{(2)}} \leq start_j^{(2)} + MaxShift_j^{(2)}$, guaranteeing the child can reach $l_j^{(2)}$ within its allowable window without re-simulating the route. The condition is checked in both directions; two cuts are sampled at random from all valid candidates. If no valid cut exists, random routes are used as fallback.

Mutation

For tailored insertion, a candidate v can be inserted before position i if $end_v + t_{v,l_i} \leq start_i + MaxShift_i$. The best candidate per position is the one maximising $p_v^3/(\Delta + 1)$, where Δ is remaining slack after insertion. The cubic exponent strongly favours high-profit landmarks; dividing by slack discourages tight insertions that would block future operators. One feasible (position, candidate) pair is chosen at random from all positions. Random deletion removes a landmark uniformly at random and never violates feasibility. With probability p_{ins} the operator inserts; otherwise it deletes. If the tour is empty, insertion is always performed.

Route resolution — GENETICSOLVER

After crossover and mutation, each child is repaired by `_resolve_tour`: while P is infeasible, remove $\arg\min_{i \in P} p_i$. Since profit is additive, removing the lowest-p_i landmark costs the least reward. This greedy repair ensures every individual entering the next population is feasible, so the running best is always valid.

Population management

Elitism preserves the top $\lfloor p_{\text{elite}} \cdot |P_t| \rfloor$ individuals unchanged each generation, preventing the best solution from being lost. Culling replaces the bottom $\lfloor p_{\text{cull}} \cdot |P_t| \rfloor$ individuals with fresh random routes, injecting diversity and reducing premature convergence. Evolution halts when no improvement is seen for `patience` consecutive generations. All hyperparameters — population size, generations, mutation rate, insertion probability, elite and cull proportions, patience — were tuned by grid search.

Additional Solvers

GRASP. GRASP bridges the constructive and local-search paradigms. Each iteration scores unvisited landmarks by $\rho_i = p_i/(t(l_{\text{cur}}, l_i) + v_i)$, builds a Restricted Candidate List of landmarks within a fraction

$\alpha = 0.3$ of the best score, selects one at random, then improves the resulting tour via Replace, Insert, and Swap operators applied in priority order. Multiple independent iterations are run and the best solution returned. The randomised construction introduces diversity while α keeps candidates competitive; v_i in the denominator prevents over-selecting long-duration landmarks that exhaust the budget with few stops.

Tabu Search. Tabu Search applies Insert, Remove, Swap, and Replace moves, pruning the neighbourhood by a weakness score (induced travel cost plus service duration, normalised by profit) to focus computation on the most promising moves. Recently accepted moves are forbidden for a fixed tenure to prevent cycling; aspiration overrides tabu status when a global best is found. Strategic oscillation alternates between intensification (hard budget T), expansion (budget relaxed to $T + \sigma$ to traverse infeasible corridors between feasible clusters), and recovery (weak landmarks removed and tabu-listed to force exploration of a genuinely new region).

CPLEX. The exact solver formulates the OPTW as a Mixed-Integer Linear Program solved by IBM CPLEX. Binary arc variables x_{ij} and visit indicators y_i model the route structure; continuous variables t_i track visit start times; MTZ constraints eliminate sub-tours and propagate timing; additional binary variables w_{ik} select the active time-window slot for each visited landmark. The model provides an optimality certificate and serves as an upper-bound reference for evaluating the heuristic solvers.

5. Application Design

The proposed system is architected as a modular, three-tier application designed to solve the **Orienteering Problem with Time Windows (OPTW)** within the urban context of Algiers. The design emphasizes the separation of concerns through a layered architecture, ensuring that the mathematical optimization logic remains decoupled from the service delivery and the user interface.

5.1. Core Logic Layer: AlgiersLIB

The foundation of the application is `AlgiersLIB`, a standalone Python library dedicated to modeling the problem domain and implementing optimization heuristics. To ensure extensibility, it utilizes the **Strategy Design Pattern**. An abstract base class, `Solver`, defines the interface for all optimization procedures, allowing the system to remain "open for extension but closed for modification."

5.2. Service Layer: Flask REST API

Built with the **Flask** framework, this layer manages data persistence, request validation, and the execution pipeline. Following the *KISS* principle, the system utilizes **CSV flat-files** for data storage, which is optimal for the static nature of the landmark dataset (≈ 50 points).

A critical performance optimization in this layer is the **Two-Phase Distance Calculation**. Because the meta-heuristics evaluate millions of potential routes, querying a real-world routing API during the search phase would be catastrophically slow and expensive. Instead, a **memoization-based caching system** computes a fast Haversine (straight-line) distance matrix for the solver loops. The Mapbox Directions API is strictly reserved for the final output phase to fetch accurate road geometries. Furthermore, a dedicated schema layer (`request_schemas.py`) ensures "Garbage In, Garbage Out" protection by validating user inputs before they reach the solvers.

5.3. Interface Layer: Frontend and UX

The frontend is developed using **Vanilla HTML5, CSS3, and JavaScript** to ensure a lightweight client-side footprint without the overhead of heavy frameworks. The UI is spatially divided to maximize usability: a left-hand configuration panel, a central interactive map canvas, a right-hand itinerary overlay, and a bottom KPI (Key Performance Indicator) dashboard.

A key feature of the interface is the preference weighting logic. The user visually ranks five specific landmark categories (**Attraction, Historical, Shopping, Religious, and Tradition/Art**). To dynamically adjust the importance of each category, the system converts the user-assigned rank (r) into a score multiplier (W_r) using an amplification coefficient ($C = 1.2$) across the total number of categories ($n = 5$):

$$W_r = \left(\frac{n-r}{n-1} - 0.5 \right) \times C + 1 \tag{1}$$

Using this formula, the ranks from 1 (best) to 5 (worst) map to a targeted multiplier spread of $\{\mathbf{1.6}, \mathbf{1.3}, \mathbf{1.0}, \mathbf{0.7}, \mathbf{0.4}\}$. These weights are passed directly to the `ScoringService` to adjust the objective function of the selected solver.

To gamify the user experience and abstract the mathematical complexity of the solvers for non-technical users, the system maps each of its optimization algorithms to a specific "Pokemon" persona reflecting its operational behavior. This roster spans simple greedy approaches (e.g., *Piplup*, *Bunnelby*), advanced meta-heuristics and evolutionary methods (e.g., *Mewtwo* for Tabu Search, *Eevee* for Genetic Algorithms), up to an exact IBM ILOG CPLEX solver (*Magnemite*) serving as the optimal baseline.

5.4. User Operational Flow

The interaction between the user and the three modules follows a deterministic workflow:

1. **Initialization:** In the left panel, the user defines the global constraints: the time window (start/end times), the starting hotel, and the specific day of the week (which enforces dynamic opening/closing hours for the OPTW constraints).
2. **Personalization:** The user ranks the five landmark categories via a drag-and-drop interface. The Frontend calculates the required weight multipliers (1.6 to 0.4) and selects a Solver.
3. **Request Orchestration:** The parameters are sent via an asynchronous AJAX request to the `/api/solve` endpoint.
4. **Backend Execution (Pipe-and-Filter Pattern):**
    - The `ProblemLoader` initializes constraints and validates the dataset.
    - The `ScoringService` applies the rank formula to compute a weighted score for every landmark.
    - The `SolverService` executes the selected strategy using the cached Haversine matrix for speed, operating under a strict **30-second maximum search time limit**.
    - The `RoutingService` fetches final road-network polylines from Mapbox for the optimized path.
5. **Visualization and Comparison:** After computation, the central Mapbox canvas renders the route. The bottom bar displays four primary KPIs: **Total Stops, Total Time, Distance (KM), and overall Interest Score**. The right-hand **Tour Guide** overlay provides a step-by-step itinerary, detailing each node's category, point value, and allocated visit duration. The solution is also added to a **Ranking Table**, allowing the user to seamlessly benchmark different algorithms.

Table 3: API Endpoint Specifications

Endpoint	Description
`/api/landmarks`	Retrieves the dataset of landmarks and metadata.
`/api/hotels`	Retrieves the list of available starting locations.
`/api/categories`	Returns landmark categories for user ranking.
`/api/solve`	Accepts constraints (budget, weights, algorithm) and returns an optimized tour.

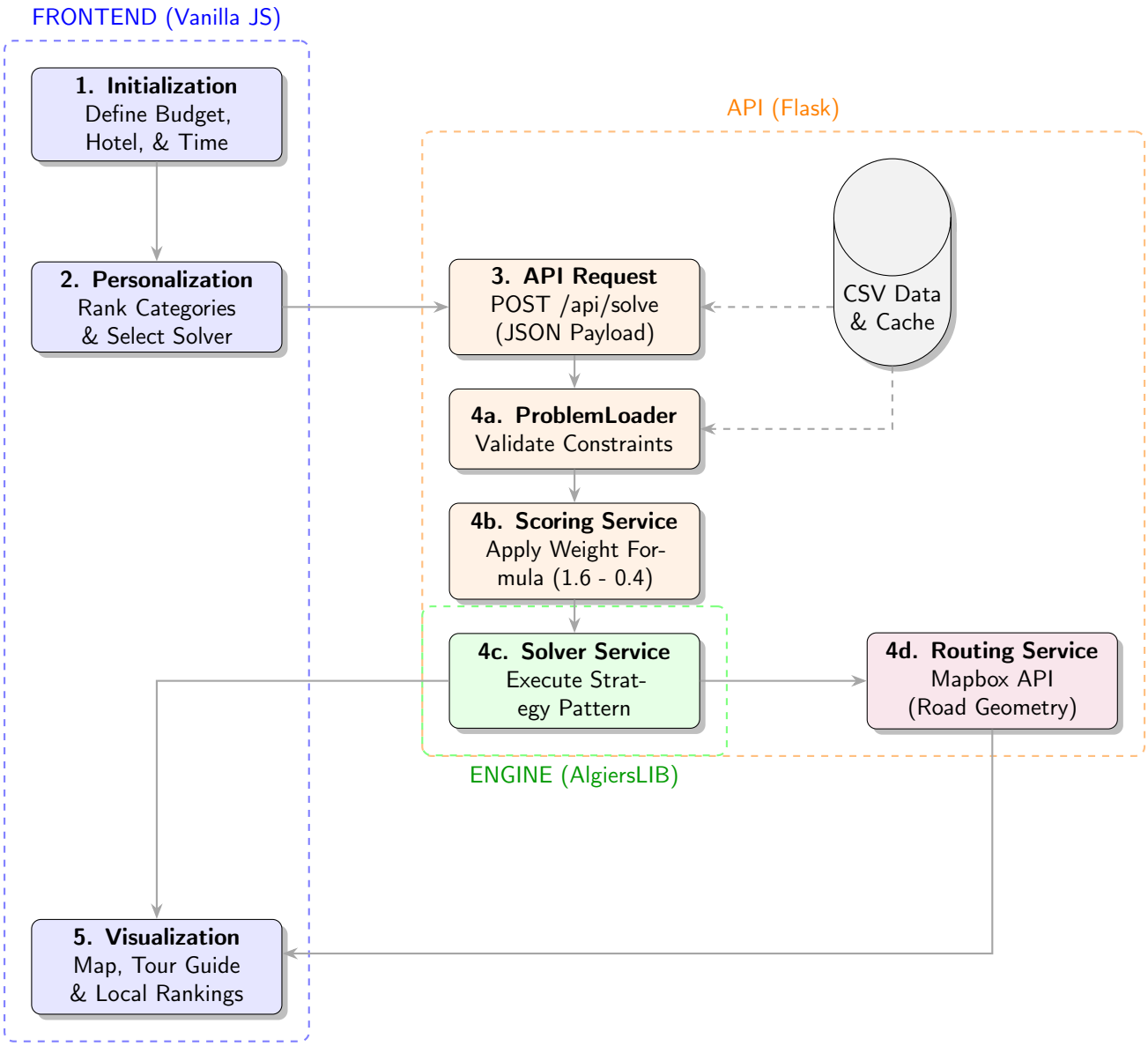

Figure 3: System Architecture of the Application.

6. Results and Analysis

6.1. Experimental Setup and Evaluation Methodology

All experiments were conducted on an AMD Ryzen 7 7435HS processor (3.10 GHz, 16 logical cores). Three datasets were used: (i) the team-collected **Algiers** dataset (59 real landmarks, multiple time windows); (ii) **Solomon-50** [18], 50-node TOPTW benchmark instances (`c/r/rc_100_50`) covering clustered, random, and mixed layouts; and (iii) **Solomon-100** [17], the full 100-node instances used for scalability stress-testing. CPLEX was excluded from Solomon-100 due to prohibitive runtime and capped at **30 s** on Algiers.

The evaluated solvers are: GREEDY (score- and ratio-priority variants), SA-BOLTZMANN, SA-CAUCHY, TABU, TABU-RANDOM, GRASP, GENETIC-TAILORED, GENETIC-SCORE, and CPLEX. Stochastic solvers ran **5 times per instance** (25 runs over 5 instances); deterministic solvers once. A multiprocessing framework with adaptive serialisation was used throughout.

Performance Metrics.

- **Score**: total collected interest along the tour.
- **Optimality gap** (%): $\text{Gap} = \max\left(0,\ \frac{\text{Optimal}-\text{Score}}{\text{Optimal}} \times 100\right)$, with CPLEX as ground truth for Solomon-50 [18] and Righini & Salani [16] optima for Solomon-100 [17].
- **Execution time** (s): wall-clock time per run.
- **Optimal-finding rate**: fraction of stochastic runs matching the known optimum.

GENETIC-SCORE is excluded from optimality comparisons as its unconstrained fitness generates infeasible tours; it is discussed in Section 6.5.

6.2. Algiers Dataset

Figure 4 presents four performance views on the Algiers dataset. GRASP and TABU both attain the highest score of **98.7** (11 landmarks, 98.2% and 99.9% budget utilisation);however TABU attains the highest score under only 0.261 s the best speed–quality trade-off on this dataset. SA-CAUCHY (96.9) and SA-BOLTZMANN (95.3) follow closely. GENETIC-TAILORED achieves 90.0 across 11 landmarks in only 0.059 s—. The GREEDY variants diverge sharply: Ratio-Priority scores 70.5 (8 landmarks) while Score-Priority collapses to 19.2 (2 landmarks), confirming that score-greedy construction fails without feasibility enforcement.

Despite being an exact solver, CPLEX reaches only 88.2 under its 30 s cap— insufficient to prove optimality on a 59-node instance with heterogeneous time windows. TABU (98.7 at 0.216 s) and all top metaheuristics surpass it, illustrating the practical advantage of local-search methods under strict time budgets.

6.3. Solomon-50: Solution Quality

Figures 5 and 6 characterise solution quality on Solomon-50 [18]. GENETIC-TAILORED achieves the lowest mean gap of **0.73%** (median 0.0%), matching the CPLEX optimum in **84%** of runs—the highest optimal-finding rate among all heuristics. GRASP attains 3.47% (60% optimal rate) with low variance. The Tabu variants (3.79% and 4.02%, rates of 48% and 40%) deliver near-optimal solutions in under 1 s—over 5× faster than GENETIC-TAILORED.

The SA variants exhibit high gaps (27.8% for SA-CAUCHY, 34.9% for SA-BOLTZMANN) and zero optimal-finding rate. Their cooling schedules—calibrated to iteration count rather than feasible neighbourhood size—cause premature convergence; the Cauchy schedule's heavier-tailed acceptance yields a ≈7 pp advantage over Boltzmann, but both remain uncompetitive. GREEDY anchors the bottom at a

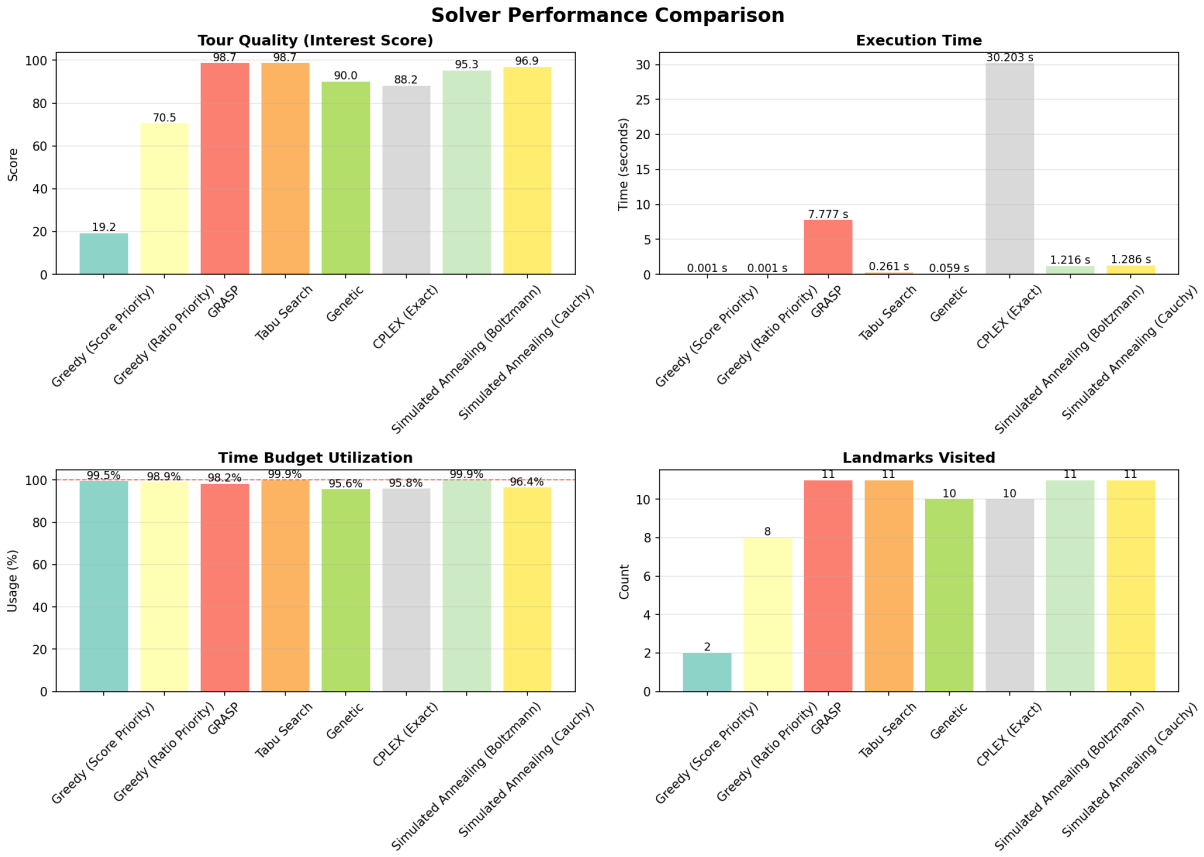

Figure 4: Solver performance on the Algiers real-world dataset (59 landmarks): tour quality (interest score), execution time, time-budget utilisation, and landmarks visited.

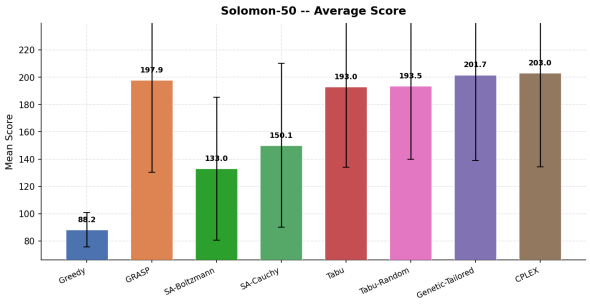

Figure 5: Score distributions on Solomon-50 [18] (box plot over 25 stochastic / 5 deterministic runs).

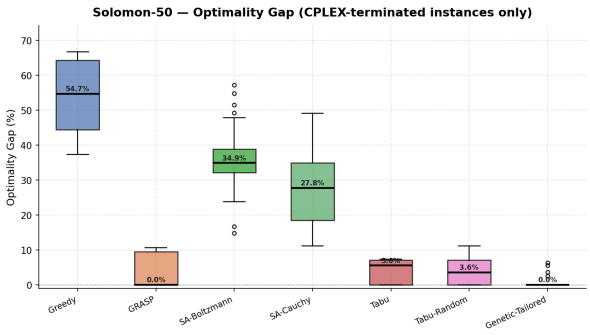

Figure 6: Optimality gap on Solomon-50 [18] (CPLEX-terminated instances only).

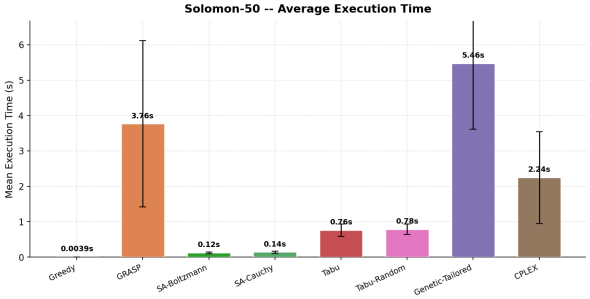

Figure 7: Mean execution time per solver on Solomon-50 [18] (error bars: $\pm 1\sigma$).

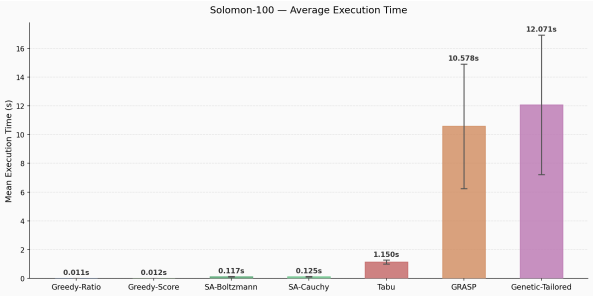

Figure 8: Mean execution time per solver on Solomon-100 [17] (error bars: $\pm 1\sigma$).

54.7% median gap.

CPLEX closes optimally on all 5 instances (avg. 2.24 s), but runtime is highly instance-dependent: easy clustered cases finish in seconds while harder ones (e.g., `c102`) can take up to **60 minutes**, rendering it impractical at scale.

6.4. Solomon-100: Scalability Analysis

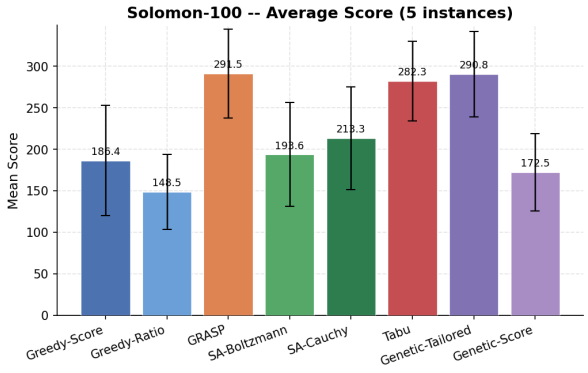

Figure 9: Score distributions on Solomon-100 [17] (25 stochastic runs per solver).

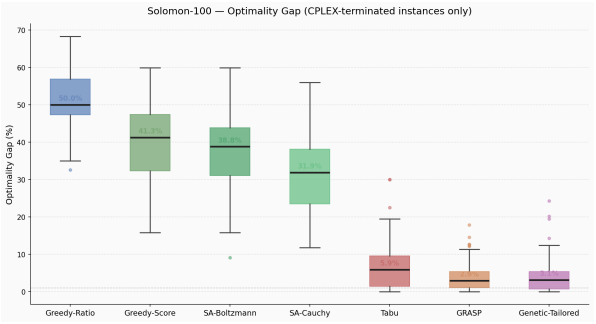

Figure 10: Optimality gap on Solomon-100 [17] (Righini & Salani [16] ground truth).

Scaling to 100 nodes (Figures 9–8), the quality ordering is preserved but gaps widen. GRASP (3.95%) and GENETIC-TAILORED (3.98%) remain statistically tied, both sustaining sub-4% gaps against the Righini & Salani [16] optima. TABU degrades modestly to 6.59% but stays the fastest viable option at 1.15 s. SA and Greedy variants deteriorate significantly: SA-CAUCHY widens to 30.6%, SA-BOLTZMANN to 37.2%, and GREEDY-RATIO reaches 51.5%.

Execution times scale non-linearly: GRASP grows to 10.58 s ($\sigma = 4.33$ s) and GENETIC-TAILORED to 12.07 s ($\sigma = 4.85$ s), while SA and Tabu variants remain below 1.2 s. Table 4 provides a unified cross-scale summary.

Table 4: Consolidated benchmark results across Solomon-50 [18] and Solomon-100 [17]. Gap on Solomon-50 is relative to CPLEX; on Solomon-100 relative to Righini & Salani [16] optima. Opt. rate computed over 25 stochastic runs (Solomon-50 only). "–" denotes not applicable. Bold: best per column.

	Solomon-50			Solomon-100		
Algorithm	**Gap (%)**	**Time (s)**	**Opt. Rate**	**Gap (%)**	**Time (s)**	**Score**
Genetic-Tailored	**0.73**	5.46	**84%**	3.98	12.07	290.8
GRASP	3.47	3.76	60%	**3.95**	10.58	**291.5**
Tabu-Random	3.79	0.78	48%	–	–	–
Tabu	4.02	**0.76**	40%	6.59	**1.15**	282.3
SA-Cauchy	27.77	0.14	0%	30.57	0.12	213.3
SA-Boltzmann	34.86	0.12	0%	37.22	0.12	193.6
Greedy-Score	53.48	0.004	0%	39.92	0.01	186.4
Greedy-Ratio	–	–	–	51.48	0.01	148.5
CPLEX	0.00	2.24	100%	–	–	–

6.5. Key Insights and Trade-off Analysis

- **Quality–time Pareto front.** Solvers span a clear frontier: Greedy and SA variants are fast but sub-optimal (<0.15 s, gaps >27%), while GRASP and GENETIC-TAILORED are near-optimal but slower

(>3.5 s, gaps <4%). TABU-RANDOM uniquely dominates the mid-range—3.79% gap in under 1 s—making it the preferred choice for latency-sensitive deployment.

- **CPLEX: exactness versus practicality.** CPLEX is an indispensable calibration tool on Solomon-50 [18], but runtime is highly non-uniform: easy instances close in seconds while `c102` can take **60 minutes**. On Algiers, its 30 s cap yields 88.2—surpassed by GENETIC (90.0), SA-BOLTZMANN (95.3), SA-CAUCHY (96.9), and both GRASP/TABU (98.7).
- **SA cooling schedules.** Both SA variants suffer premature convergence because cooling is tied to iteration count rather than feasible neighbourhood size. Cauchy outperforms Boltzmann by ≈7 pp via heavier-tailed acceptance, but gaps grow with scale and neither variant ever finds the optimum.
- **GRASP vs. Genetic-Tailored.** Statistically indistinguishable gaps at both scales (<0.1 pp on Solomon-100 [17]). GENETIC-TAILORED leads in optimal-finding rate (84% vs. 60% on Solomon-50 [18]); GRASP is simpler and trivially parallelisable. The choice depends on engineering effort and the required frequency of finding the optimum.
- **Fitness design in constrained GAs.** GENETIC-SCORE's unconstrained fitness caused all Algiers runs and 76% of Solomon-50 [18] runs to exceed the time budget, with scores up to 4× the CPLEX optimum (e.g., 970 vs. 180 on `rc101`). This is a structural failure: feasibility enforcement within the fitness function is a prerequisite for GAs on hard-constrained problems.
- **Instance-class sensitivity.** GENETIC-TAILORED leads on Solomon-50 [18]; GRASP leads on Solomon-100 [17]; both tie on Algiers. Clustered instances favour crossover-based search; random instances reward GRASP's greedy-randomised construction. This motivates instance-aware or hybrid solvers.

7. Discussion

The experimental results reveal a nuanced performance landscape in which solution quality, computational cost, and constraint satisfaction trade off in ways that both confirm and extend the findings reported in the orienteering literature.

7.1. Interpretation of Results

The most striking result is the parity between GRASP and Tabu Search at the highest observed tour quality of 98.7, both visiting 11 landmarks within the eight-hour budget, yet with a wall-clock time ratio of approximately 27:1 in favour of Tabu Search (0.37 seconds versus 10.18 seconds). This finding is consistent with the comparative analysis of Liang, Kulturel-Konak, and Smith [11], who observed that Tabu Search and population-based heuristics achieve comparable solution quality on orienteering benchmarks, and with Tang and Miller-Hooks [19], who demonstrated that adaptive memory mechanisms in Tabu Search enable rapid convergence through structured neighbourhood pruning. The successive-filtering candidate strategy implemented in our Tabu Search — ranking landmarks by induced travel cost relative to interest score — appears to be the principal factor responsible for its computational efficiency, effectively reducing the neighbourhood evaluation burden without sacrificing solution quality.

Simulated Annealing with the Cauchy acceptance criterion (96.9, 11 landmarks) consistently outperforms the Boltzmann variant (95.3, 11 landmarks), a result attributable to the heavier tails of the Cauchy distribution, which sustain a higher probability of accepting deteriorating moves at intermediate temperatures and thereby reduce premature convergence. This echoes the theoretical expectations discussed by Gunawan, Lau, and Vansteenwegen [6], who note that acceptance-function design is a primary differentiator of Simulated Annealing variants on constrained routing problems.

The Genetic Algorithm (68.3–90.0 across configurations) exhibits greater variance than single-solution methods, which is consistent with the sensitivity of population-based approaches to crossover operator design on time-windowed instances. The order crossover and tailored crossover implementations produced different fitness landscapes, and the relatively modest performance of the penalty fitness function relative to the feasibility-oriented variant suggests that soft-constraint handling significantly affects search trajectory for this problem class.

The most academically interesting result is the underperformance of the CPLEX exact solver (61.8–88.2, 7–10 landmarks) relative to GRASP and Tabu Search. Three explanations are plausible. First, CPLEX required 30.2 seconds to terminate, suggesting that the branch-and-bound search tree was not fully explored within the available computation budget, and the best integer solution found at termination is consequently a feasible but sub-optimal incumbent. Second, the Mixed Integer Linear Programming formulation introduces symmetry constraints and big-M linearisations for time-window propagation that tighten the feasible region relative to the heuristic representations. Third, the haversine-based travel matrix, while consistent across all solvers, produces a non-integer cost structure that may slow LP relaxation bounds. Righini and Salani [15] observed similar behaviour and addressed it through decremental state-space relaxation; incorporating such techniques would likely close the gap between CPLEX and the leading metaheuristics.

7.2. Limitations

Several limitations constrain the generalisability of these results. First, travel times are computed from straight-line haversine distances at a fixed average urban speed of 25 km/h. Real road distances in Algiers, which the backend routing service retrieves from MAPBOX API, are systematically longer and temporally variable due to traffic. The algorithms were therefore optimising a surrogate objective that diverges from the true tourist experience. Second, landmark interest scores were assigned through a weighted combination of public ratings and expert judgement, a process that introduces subjectivity and limits reproducibility. Third, the dataset is confined to 56 landmarks in the Algiers wilaya, a scale at

which all tested algorithms remain tractable; results may not extrapolate to larger metropolitan datasets. Finally, the time-window representation assumes deterministic opening hours, whereas real tourist sites may close unexpectedly or have dynamic crowd-dependent effective visit durations.

7.3. Future Work

Several directions offer meaningful improvements. Within the classical metaheuristic paradigm, Adaptive Large Neighbourhood Search (ALNS) represents the current state of the art for time-constrained orienteering [6], with its self-tuning destroy-and-repair operators addressing the principal weakness of both GRASP (construction diversity) and Tabu Search (neighbourhood breadth). Iterated Local Search with the MaxShift data structure of Vansteenwegen et al. [22] would provide an efficient feasibility oracle that reduces the per-iteration cost of neighbourhood evaluation from $O(n)$ to $O(1)$.

At the frontier of the field, deep reinforcement learning (DRL) approaches have demonstrated near-optimal performance on routing problems of this scale. Kool, Hoof, and Welling [9] showed that an Attention Model trained with the REINFORCE policy gradient algorithm learns generalisable construction heuristics for TSP and related problems; Gama and Fernandes [4] extended this framework directly to the OPTW. Hybrid DRL-ALNS architectures, in which a learned policy selects neighbourhood operators dynamically [13], represent the most promising direction for achieving both solution quality and sub-second inference times. A further extension of direct practical relevance is the Stochastic Orienteering Problem, in which travel times are drawn from a distribution reflecting real traffic conditions; this formulation is underexplored in the literature [21] and maps naturally onto the Algiers urban context. Finally, extending the single-day formulation to the Team Orienteering Problem with Time Windows [6] would enable multi-day, multi-guide itinerary planning, substantially increasing the practical utility of the system.

8. Conclusion

This paper presented *Algiers in 24 Hours*, a tourist route optimisation system grounded in the Time-Constrained Orienteering Problem with Time Windows. Six algorithmic approaches spanning the full spectrum from deterministic construction to exact branch-and-bound were implemented, integrated into a common model layer, and evaluated on a curated dataset of 56 Algiers landmarks with verified GPS coordinates, opening hours, and interest scores.

The principal finding is that Tabu Search with Strategic Oscillation achieves solution quality equivalent to GRASP (98.7 interest score, 11 landmarks) at a fraction of the computational cost (0.37 versus 10.18 seconds), establishing it as the recommended solver for real-time deployment contexts. Simulated Annealing with the Cauchy acceptance criterion provides a competitive alternative (96.9) with moderate runtime. The underperformance of CPLEX relative to leading metaheuristics underscores the known intractability of the OPTW for branch-and-bound approaches under realistic time budgets, and motivates the design of hybrid exact-heuristic methods as a direction for future investigation.

Beyond algorithmic contribution, this work produces a reusable, open-source problem model class for the Algerian tourist orienteering context — a domain absent from existing benchmark libraries — and a full-stack web application through which algorithm behaviour can be explored interactively by non-expert users. The gap between the best heuristic score and the CPLEX incumbent across all experiments defines a concrete quality target for future work. Closing that gap through Adaptive Large Neighbourhood Search or learned construction heuristics, while extending the formulation to multi-day and stochastic settings, constitutes the principal agenda for subsequent research.

References

[1] C. Archetti, M.Grazia Speranza, and Alain Hertz. "A Tabu Search Algorithm for the Split Delivery Vehicle Routing Problem". In: *Transportation Science* 40 (Nov. 2006), pp. 64–73. DOI: 10.1287/trsc.1040.0103.

[2] Thomas A. Feo and Mauricio G. C. Resende. "Greedy Randomized Adaptive Search Procedures". In: *Journal of Global Optimization* 6.2 (Mar. 1, 1995), pp. 109–133. ISSN: 1573-2916. DOI: 10.1007/BF01096763. URL: https://doi.org/10.1007/BF01096763 (visited on 05/16/2026).

[3] Matteo Fischetti, Juan José Salazar González, and Paolo Toth. "Solving the Orienteering Problem through Branch-and-Cut". In: *INFORMS Journal on Computing* 10.2 (May 1998), pp. 133–148. ISSN: 1091-9856. DOI: 10.1287/ijoc.10.2.133. URL: https://pubsonline.informs.org/doi/10.1287/ijoc.10.2.133 (visited on 05/16/2026).

[4] Rodrigo Gama and Heli Fernandes. "A reinforcement learning approach to the orienteering problem with time windows". In: *arXiv preprint arXiv:2010.05436* (2020). URL: https://arxiv.org/abs/2010.05436.

[5] Bruce Golden, L. Levy, and Rakesh Vohra. "The orienteering problem". In: *Nav Res Logist* 34 (June 1987), pp. 307–318. DOI: 10.1002/1520-6750(198706)34:3<307::AID-NAV3220340302>3.0.CO;2-D.

[6] Aldy Gunawan, Hoong Chuin Lau, and Pieter Vansteenwegen. "Orienteering problem: A survey of recent variants, solution approaches and applications". In: *European Journal of Operational Research* 255.2 (2016), pp. 315–332. DOI: 10.1016/j.ejor.2016.04.059.

[7] Marisa G. Kantor and Moshe B. Rosenwein. "The Orienteering Problem with Time Windows". In: *The Journal of the Operational Research Society* 43.6 (1992), pp. 629–635. ISSN: 01605682, 14769360. URL: http://www.jstor.org/stable/2583018 (visited on 05/16/2026).

[8] Joanna Karbowska-Chilinska and Paweł Zabielski. "A Genetic Algorithm vs. Local Search Methods for Solving the Orienteering Problem in Large Networks". In: vol. 7828. Sept. 2012. ISBN: 978-3-642-37342-8. DOI: 10.1007/978-3-642-37343-5_2.

[9] Wouter Kool, Herke van Hoof, and Max Welling. "Attention, Learn to Solve Routing Problems!" In: *7th International Conference on Learning Representations (ICLR)*. 2019. URL: https://openreview.net/forum?id=ByxBFsRqYm.

[10] Nacima Labadie et al. "The Team Orienteering Problem with Time Windows: An LP-based Granular Variable Neighborhood Search". In: *European Journal of Operational Research* 220.1 (2012), pp. 15–27. ISSN: 0377-2217. DOI: https://doi.org/10.1016/j.ejor.2012.01.030. URL: https://www.sciencedirect.com/science/article/pii/S0377221712000653.

[11] Yun-Chia Liang, Sadan Kulturel-Konak, and Alice E. Smith. "Meta heuristics for the orienteering problem". In: *Proceedings of the 2002 Congress on Evolutionary Computation (CEC 2002)*. Vol. 1. 2002, pp. 384–389. DOI: 10.1109/cec.2002.1006265.

[12] Roberto Montemanni and Luca Maria Gambardella. *An Ant Colony System for the Team Orienteering Problem with Time Windows*. 2023. arXiv: 2305.07305 [math.OC]. URL: https://arxiv.org/abs/2305.07305.

[13] Rik Reijnen et al. "Operator selection in adaptive large neighbourhood search using deep reinforcement learning". In: *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*. 2024.

[14] Giovanni Righini and Matteo Salani. "Decremental state space relaxation strategies and initialization heuristics for solving the Orienteering Problem with Time Windows with dynamic programming". In: *Computers & Operations Research* 36.4 (2009), pp. 1191–1203. ISSN: 0305-0548. DOI: https://doi.org/10.1016/j.cor.2008.01.003. URL: https://www.sciencedirect.com/science/article/pii/S030505480800004X.

[15] Giovanni Righini and Matteo Salani. “Decremental state-space relaxation strategies and initialization heuristics for solving the orienteering problem with time windows with dynamic programming”. In: *Computers & Operations Research* 36.4 (2009), pp. 1191–1203. DOI: 10.1016/j.cor.2008.01.003.

[16] Giovanni Righini and Matteo Salani. *Optimal solutions for the TOPTW – benchmark instances*. https://www.mech.kuleuven.be/en/mim/op/RighiniSalani.

[17] *Solomon TOPTW Instances – 100 nodes*. https://www.mech.kuleuven.be/en/mim/op/instances/righiniTOPTW2. KU Leuven, MIM Operations Research.

[18] *Solomon TOPTW Instances – 50 nodes*. https://www.mech.kuleuven.be/en/mim/op/instances/righiniTOPTW1. KU Leuven, MIM Operations Research.

[19] Hui Tang and Elise Miller-Hooks. “A tabu search heuristic for the team orienteering problem”. In: *Computers & Operations Research* 32.6 (2005), pp. 1379–1407. DOI: 10.1016/j.cor.2003.11.008.

[20] Theodore Tsiligiridis. “Heuristic Methods Applied to Orienteering”. In: *Journal of the Operational Research Society* 35 (Sept. 1984), pp. 797–809. DOI: 10.1057/jors.1984.162.

[21] Pieter Vansteenwegen, Wouter Souffriau, and Dirk Van Oudheusden. “The orienteering problem: A survey”. In: *European Journal of Operational Research* 209.1 (2011), pp. 1–10. DOI: 10.1016/j.ejor.2010.03.045.

[22] Pieter Vansteenwegen et al. “Iterated local search for the team orienteering problem with time windows”. In: *Computers & Operations Research* 36.12 (2009), pp. 3281–3290. DOI: 10.1016/j.cor.2009.03.008.

A. Screenshots of the System

Appendix A: Application Screenshots

The following screenshots illustrate the web application developed as part of this project, covering the main interface, configuration panels, solver execution, and result visualisation.

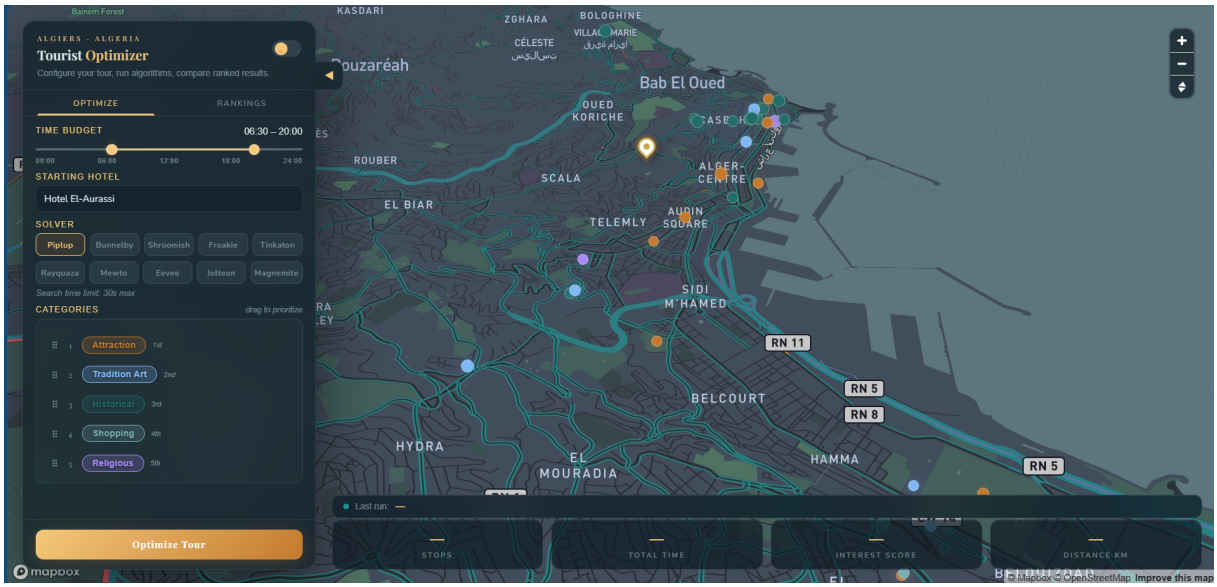

Figure 11: Main interface of the tourism itinerary planning application in dark mode.

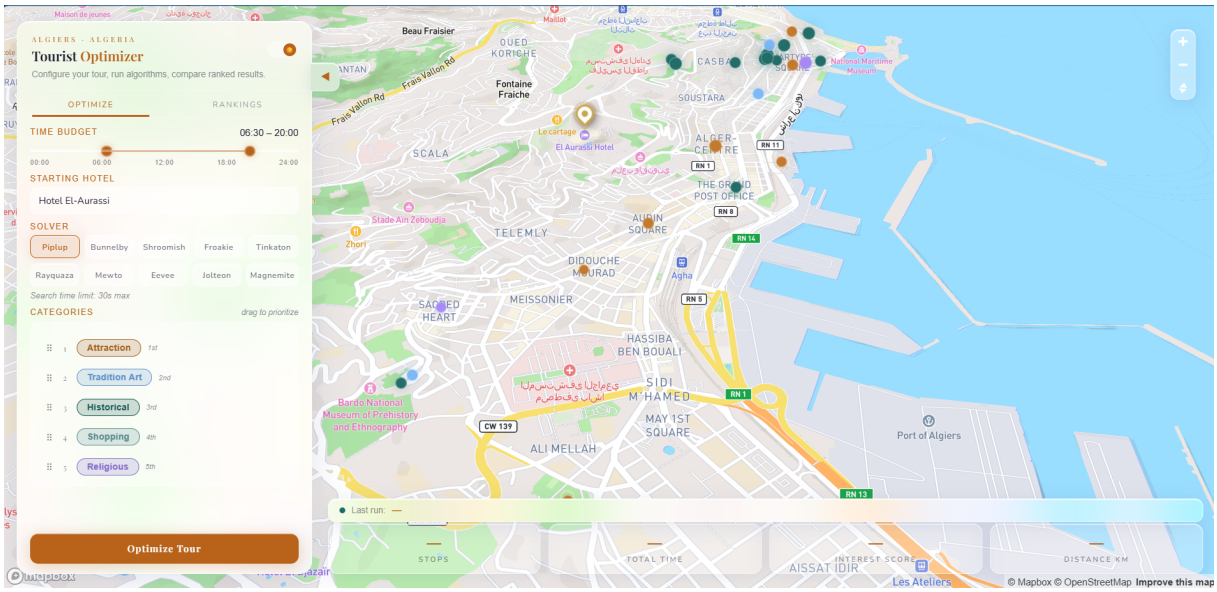

Figure 12: Light mode variant of the application interface, offering an alternative visual theme.

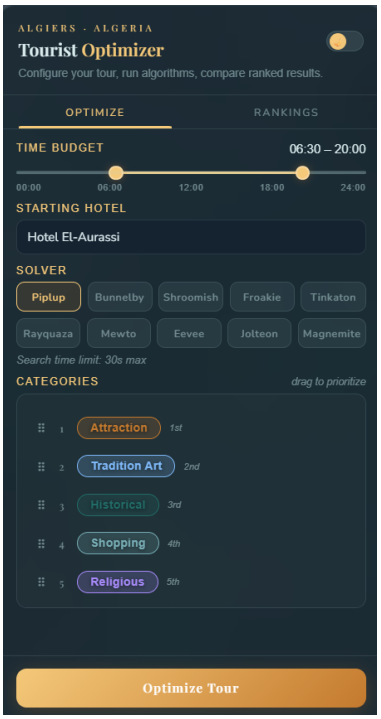

Figure 13: Control panel for configuring the time budget, travel speed, and departure point.

Figure 14: Drop-down menu for selecting the starting hotel or departure location.

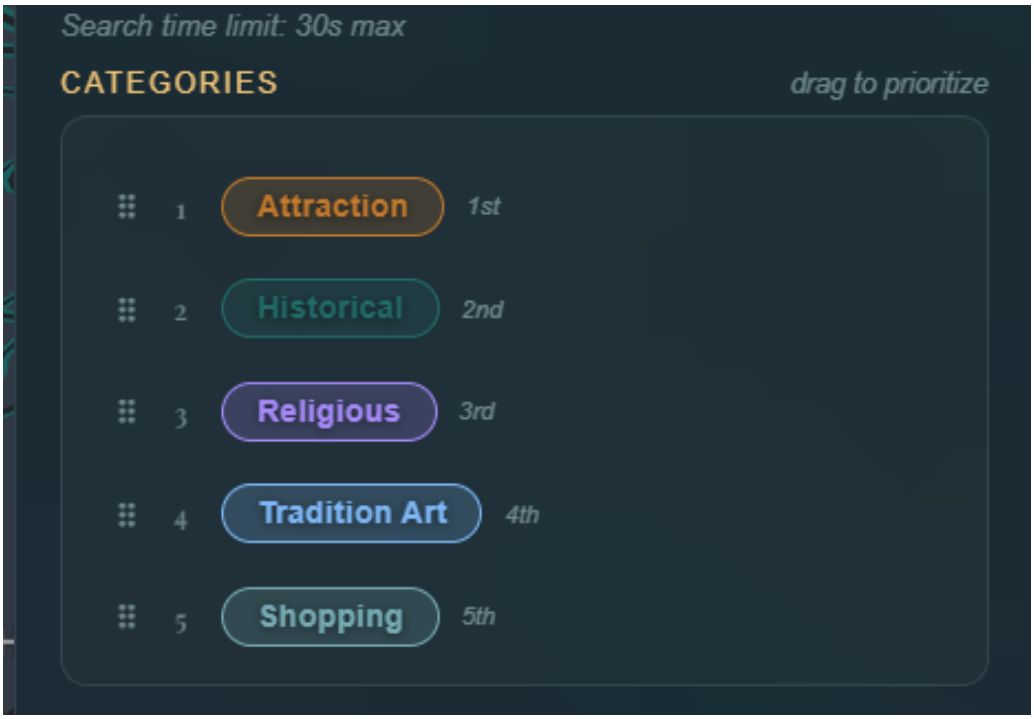

Figure 15: Drag-and-drop interface for reordering priority among landmark categories.

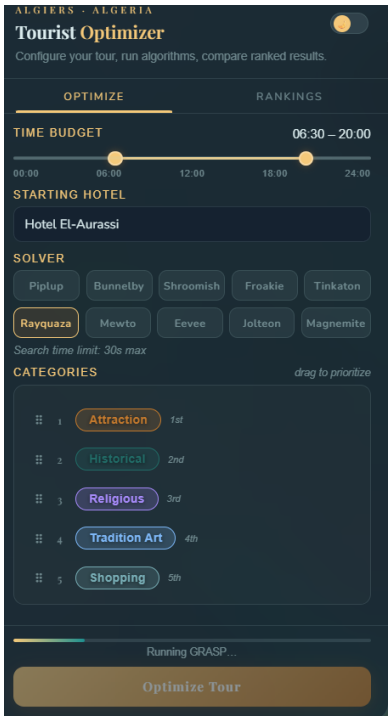

Figure 16: Solver selection panel listing the available optimisation algorithms to execute.

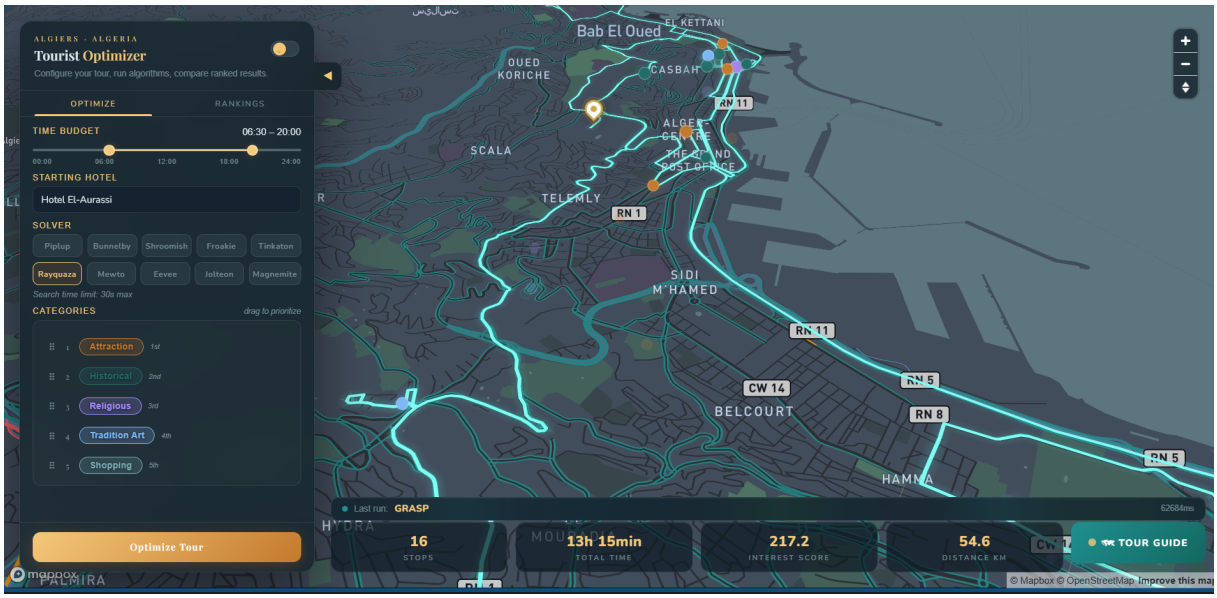

Figure 17: Optimised tour route computed by GRASP, rendered on an interactive map.

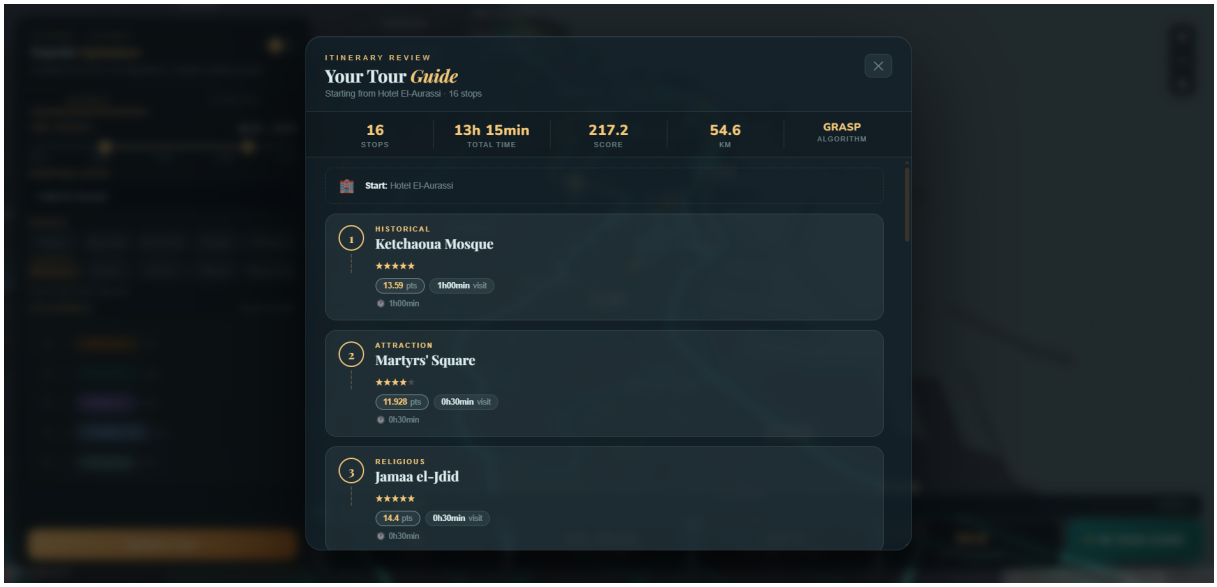

Figure 18: Detailed summary of the best tour found by GRASP: visited landmarks, scores, and timing.

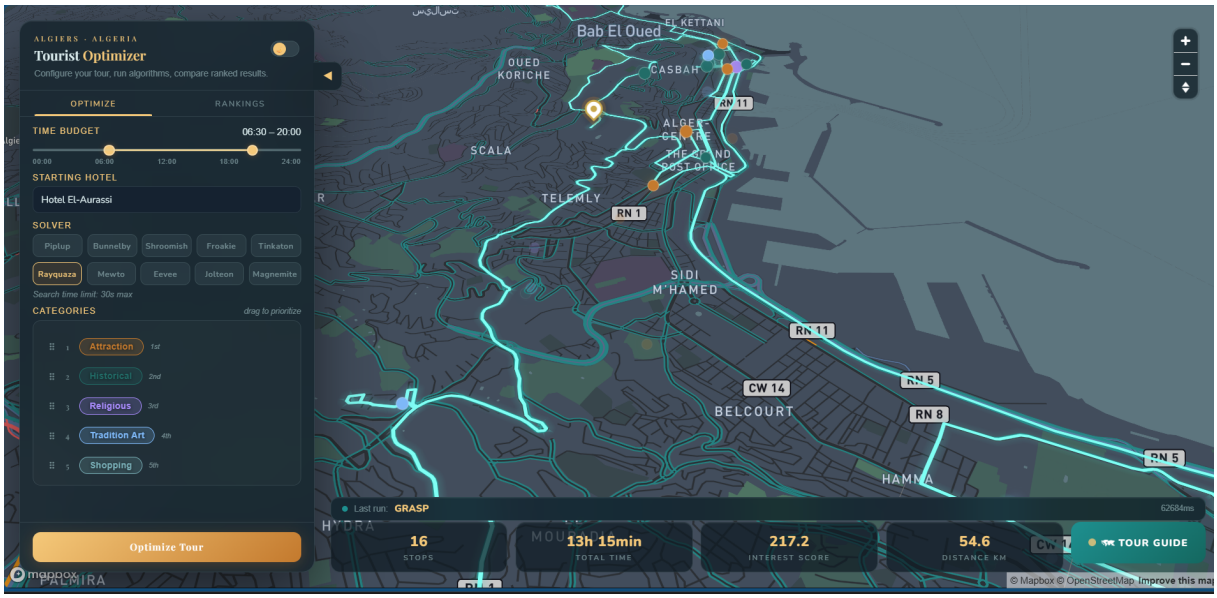

Figure 19: Full map view of the GRASP-computed itinerary with route path overlay.

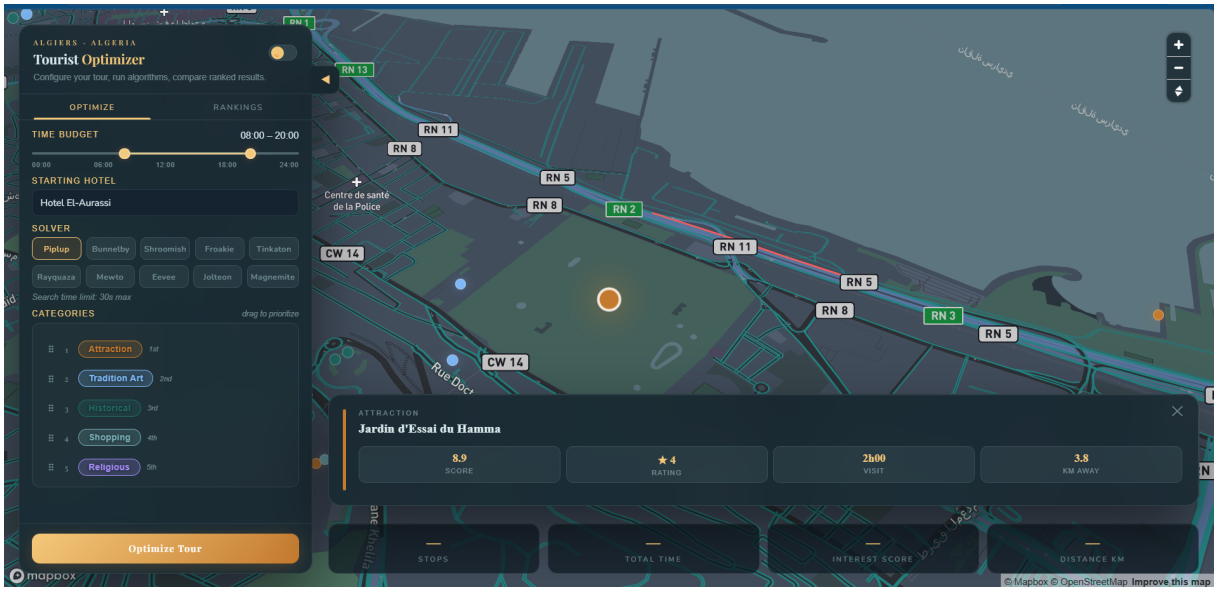

Figure 20: Landmark information tooltip displayed on map hover: name, category, and time window.

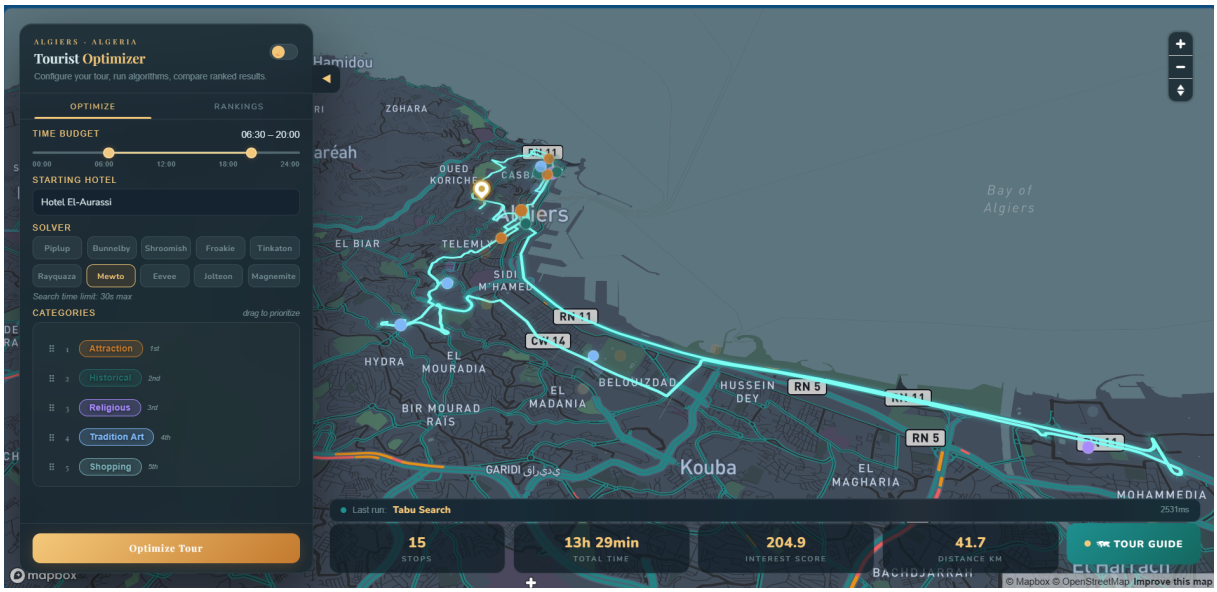

Figure 21: Optimised itinerary produced by Tabu Search displayed on the map.

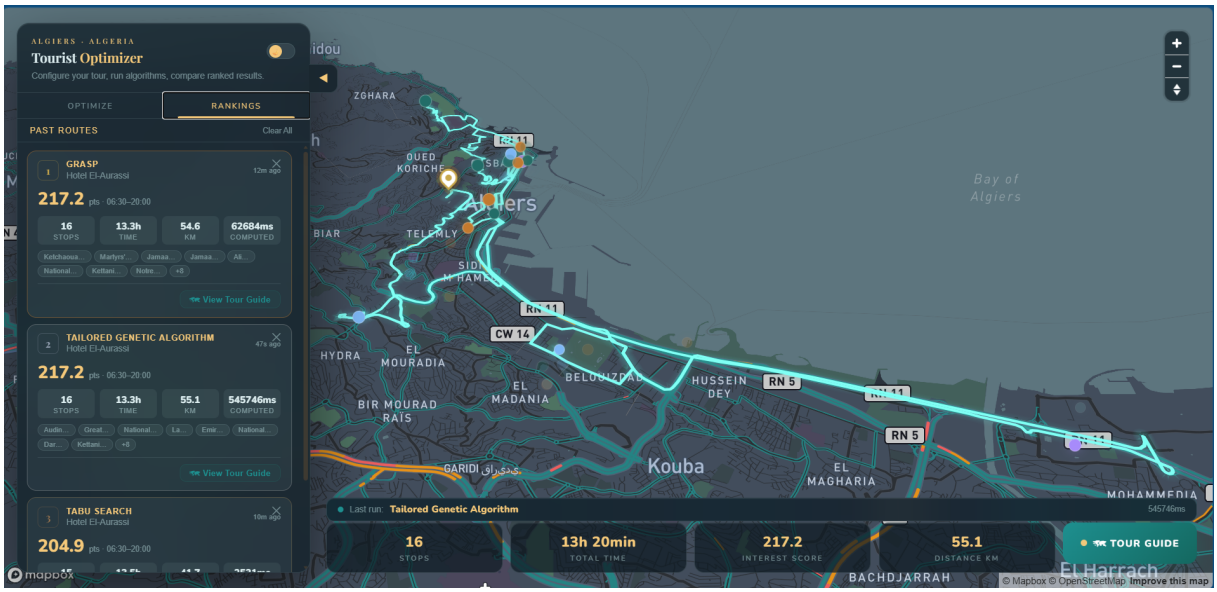

Figure 22: Algorithm performance ranking after multiple solver runs on the selected instance.

B. Who Did What

Table 5: Team member contributions.

Member	Primary Contributions
Raouf Ould Ali	Integrate Frontend, Implement Simulated Annealing, Implement CPLEX Solution, Setup GitHub Repository, Review Performance Analysis, Review Grasp Search, Write Application Design (Report), Review Dataset Building (Report), Review Conclusion (Report), Write References (Report).
Mohamed Zakaria Razam	Implement Frontend, Implement Greedy Search Algorithm, Write Code Documentation, Review Simulated Annealing, Review CPLEX Solution, Write Dataset Building (Report), Review Application Design (Report), Review Discussion (Report), Write Appendix B (Report).
Ahmed Adnane Meddah	Review Frontend, Implement Performance Analysis & Visualization, Implement Unit Testing, Review Greedy Search, Write Introduction (Report), Review Appendix A (Report), Review Appendix B (Report), Review Jupyter Notebook.
Mohamed Charaf Eddine Deghboudj	Backend Development, Implement Problem Model Class, Implement Grasp Search, Review Genetic Algorithm, Review Tabu Search, Review State of the Art (Report), Review Results and Analysis (Report), Write Discussion (Report), Write Conclusion (Report).
Yacine Mouloud	Backend Development, Implement Problem Model Class, Implement Tabu Search, Implement Unit Testing, Review Abstract (Report), Review Problem Solving Techniques (Report), Write Results and Analysis (Report), Write Appendix A (Report).
Aboubakr Bendahmane	Implement Genetic Algorithm, Setup LaTeX Template, Review Problem Model, Review Unit Testing, Write Abstract (Report), Write State of the Art (Report), Write Problem Solving Techniques (Report), Review Introduction (Report), Review References (Report).